DevOps

Other Topics

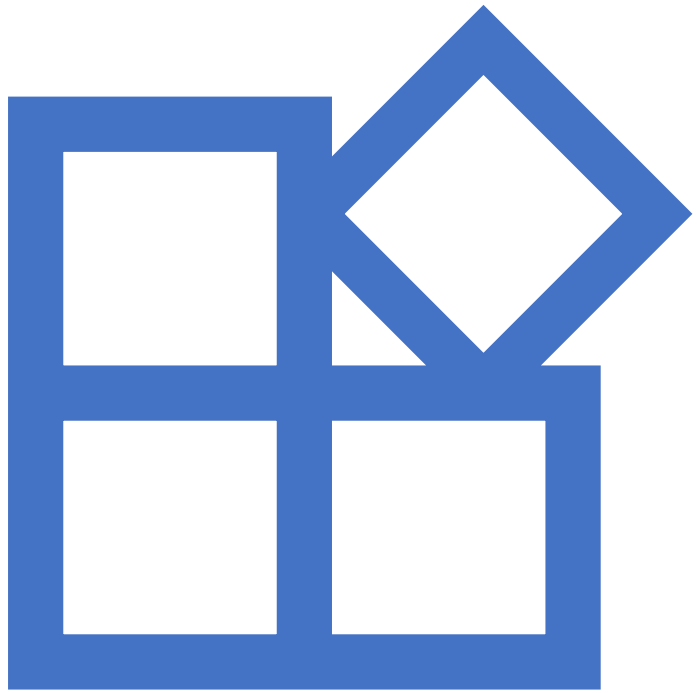
Bibliography

Laszewski, Tom; Arora, Kamal; Farr, Erik; Zonooz, Piyum. Cloud Native Architectures: Design high-availability and cost-effective applications for the cloud (p. 8). Packt Publishing

Containers and serverless

Most common way to build microservices

Allow for systems to be designed and deployed at scale and with agility that typically comes with microservices



Containers

Containers

Natural progression of additional isolation and virtualization of hardware

A virtual instance allow for a bare metal server to have multiple hosts => increase efficiencies and usage of server's resources

Containers much more lightweight, portable, and scalable

- Unused components are removed
- Only startup, shutdown and task management

Deploying using containers

Developers include specific libraries or languages and deploy the code and configurations directly to the container file

Container files are treated as deployment artifact and published to a registry to be deployed or updated across the fleet of containers already running in the cluster via the orchestration service

Containers are scaled across many host nodes maintained and scheduled by the orchestration service

File storage: local ephemeral filesystems or persistent volume mounted and accessible to all containers on a node

Containers can be grouped in pods => instantiated and scaled together

Registries

Public or private area for the storage, and serving, of pre-built containers

Major cloud vendors have a hosted version of registries that can be made private for a specific organization

Docker Hub registry – holds publicly available pre-built container files that have lots of common configurations ready to use

In production – hosted private registries

If hosted in the cloud environment, this also provides close proximity to the orchestration service for a faster deployment pipeline

Registries and CI/CD

Push in a repository (GIT) triggers a build (e.g., Azure DevOps)

Build the container and test it

Push the container to the registry

Orchestration tool pulls the container from registry

Perform an update or deploy – rolling update or new deployment

Orchestration

Problem: as a subsystem is broken down into many separate primitive microservices, the ability to coordinate those services, made up of one or many containers, becomes very difficult

Deployment of new containers to the cluster is critical and usually needs to be done with little or no downtime, making a service that performs those updates very valuable

Cloud vendor's container orchestration service are preferred over a third-party tool

Kubernetes most used container orchestration solution

Cloud provider offer managed Kubernetes services

Kubernetes Key Concepts

Kubernetes master: This is responsible for maintaining the desired state for the cluster.

Kubernetes node: Nodes are the machines (VMs, physical servers, and so on) that run the applications and cloud workflows. The Kubernetes Master controls each node; you'll rarely interact with nodes directly.

Pod: This is the basic building block of Kubernetes; the smallest and simplest unit in the Kubernetes object model that you create or deploy. A pod represents a running process on your cluster.

Kubernetes Key Concepts

Service: An abstraction that defines a logical set of pods and a policy by which to access them, sometimes called a microservice.

Replica controllers and ReplicaSets: These ensure that a specified number of pod replicas are running at any one time. In other words, they make sure that a pod or a homogeneous set of pods is always up and available.

Deployment: Provides declarative updates for pods and ReplicaSets.

DaemonSet: Ensures that all (or some) nodes run a copy of a pod. As nodes are added to the cluster, pods are added to them. As nodes are removed from the cluster, those pods are garbage collected. Deleting a DaemonSet will clean up the pods it created.

Containers key features

Native cloud services offered by the vendor

Extreme automation through CI/CD

Great for microservices

Containers usage patterns

Microservices with containers

Contain exactly what is needed, with no additional overhead to slow down the processing

Develop a specific service using containers, with desired programming language, libraries, API implementation style, and scaling techniques

Keep small groups agile and moving fast

Allows for the architecture standards board to implement guidelines that all services must follow without impacting the agility of the design team

Hybrid and migration of application deployment

Containers can be deployed in the cloud or on-premise

Deploy full composite application to on-premises and use the cloud environment to be the disaster recovery (DR) or failover site, or vice versa

Containers are a great way to run a hybrid architecture or to support the migration of a workload to the cloud faster and easier than with most other methods

Container anti-patterns

Using containers for multiple concerns

Each module should have responsibility over a single part of functionality

Breaking down monoliths into discrete components will ensure the applications have a lower blast radius and are easier to deploy and scale independently

Treating containers as VMs

Containers should be set up and configured part of the initial deployment phase

immutable component

Serverless

Serverless

Resources can be used without having to consider server capacity
No need to provision, scale, or maintain servers
Focus on developing the application
Pricing done in units of 100ms
Good match for microservices
Most common use is Function as a Service

Areas that have serverless services


Compute


API proxy


Storage


Data Stores


Inter process messaging


Orchestration


Analytics


Developer tools

Scaling

- Able to scale by design
- Consumption units are the important factor for these services, typically throughput or memory, which can be automatically updated to have more or less capability as the application requires
- Functions are design to execute as an event => scalability of the functions
- Functions are design to be triggered often (1000x / second)

Serverless usage patterns

Web and backend app processing

Data and batch processing

System automation

Web and backend application processing

Traditional three-tier application architecture does not fully take advantage of the cloud native services and needs capacity planning

Serverless alternative:

- Static frontend page is hosted in an object store (Amazon S3 or Amazon Blob Storage) – no dedicated web server
- API call made to an API Gateway (Amazon API Gateway or Azure API Management) – scalable entry point – dynamically execute user request through a Lambda Function / Azure Function (event per API call)
- Function will update Dynamo DB / Azure SQL
- Other backend functionalities can be performed by forked functions (putting a message into a queue/topic)

Data and batch processing - Real-time processing and analytics

Stream or clicks streams comes via streaming data service (Amazon Kinesis or Azure Stream Analytics), which executes a serverless function (Lambda, Azure Function) on each micro batch of data that is processed or analyzed

The serverless function stores the important information (sentiment analysis, popular ad clicks) in a DynamoDB / Azure Table Storage that can trigger other serverless processing

Data and batch processing - Batch data processing

File uploaded to object store (Amazon S3, Azure Blob Storage)

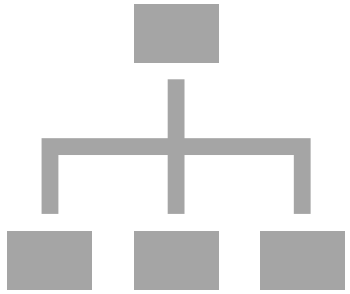
The event triggers a serverless function (Lambda, Azure Function) that performs the operation

Any data can be stored to other data stores (DynamoDB / Azure Table Storage)

A message is put in a queue => notification sent to user

System automation

Perform common environment maintenance and operations



Critical tasks to support that growth stay consistent and do not require the organization to linearly scale up the operations team supporting the environment

System automation - Common administration use cases

Setup a function that execute on a schedule and checks

- every instance across all accounts for proper tagging
- correctly sized instance resource
- stopping unused instances
- cleaning up unattached storage devices

Security administration

- Verify encryption of objects put in storage
- Verify account user policies are correctly apply
- Perform checks against custom compliance policies

Serverless anti patterns and concerns

Long-running requests (functions should be short < 5min.) – better to use containers

Event source executes in a manner that is not consistent with expectation => function triggered more than expected => increased cost and possibly data corruption or loss

Test event trigger

Setup alerts

Is important to know how the services fit together and when to invoke a function and when a custom solution

Demo Azure Function

- https://learn.microsoft.com/en-us/azure/azure-functions/functions-overview?pivots=programming-language-csharp

- Triggers
- https://learn.microsoft.com/en-us/azure/azure-functions/functions-triggers-bindings?tabs=isolated-process%2Cpython-v2&pivots=programming-language-csharp

Demo Azure Container Apps (managed Kubernetes cluster)

- https://learn.microsoft.com/en-us/azure/container-apps/overview

- Keda
- https://keda.sh/
- https://keda.sh/docs/2.14/scalers/

- Dapr
- https://dapr.io/

Hosting Azure Functions in Azure Containers Apps

- https://learn.microsoft.com/en-us/azure/azure-functions/functions-container-apps-hosting

Container Options in Azure

- https://learn.microsoft.com/en-us/azure/container-apps/compare-options

Thank you